UNIVERSITY OF SOUTHEASTERN PHILIPPINES

Real-Time Hand Motion Detection for

AR Motion Smasher

A Computer Vision-Based Gesture Recognition System for Mobile Augmented Reality Gaming

A PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR

ICE 323 Professional Elective 3

SUBMITTED BY:

Crucio, John Paul S.

Micaroz, Arthur Dale Enrique

Renigado, Kyle Harvey C.

SUBMITTED TO:

Ms. Luchi Dela Cruz

APRIL 2026

Table of Contents

I. Cover Page **1**
- i. Project Title 1
- ii. Group members 1

II. Table of Contents **2**
III. Introduction **3**
IV. Problem Statement **4**
V. Objectives **5**
- i. General Objectives 5
- ii. Specific Objectives 5

VI. Proposed Solution **6**
VII. Scope and Limitations **7**
- i. Scope 7
- ii. Limitations 7

VIII. Methodology **9**
- i. System Workflow 9
- ii. Image Processing Techniques to be used 10
- iii. Tools and Technology 11

IX. Dataset/Data Collection **12**
X. System Design **13**
XI. References **16**

III. Introduction

The rapid expansion of mobile games has resulted in the creation of a new category of entertainment with significant touch screen usage on mobile devices. However, with advances in image processing and computer vision, there are now numerous possibilities to create digitally rich experiences that provide more interactivity and immersion through physical interaction. This document provides a roadmap to build a real-time hand detection system, which will act as the central image-processing platform for an augmented reality (AR) mobile game, to be called AR Motion Smasher, on the Android operating system. The image-processing component of AR Motion Smasher will be responsible for detecting the player's hand or their brightly colored markers using the live video feed from the device's camera and the application processing of each frame of video in real time, and using Python and OpenCV to detect and provide the coordinates of the hand/marker by returning the X/Y centroid of that hand/marker position via color segmentation.In the Android gaming engine, the X/Y coordinates can be used to implement collision detection and allow users to play without the use of a controller or other devices; instead, users will use their body actions to perform the game moves. These developments in coding and programming are indicative of the intersection between HCI, mobile apps, and the growth of exergames, which are physically active video games designed to address inactivity by adding an element of exercise to video games.

IV. Problem Statement

Mobile games are generally about using a mobile device's screen to passively interact with game occurrences and not being physically involved in the activity. The main groups of people who play mobile games include children, students and recreational players. Research from multiple countries suggest that these groups are experiencing negative health consequences as a result of limited physical activity associated with playing mobile games, such as obesity, musculoskeletal pain and reduced cardiovascular fitness (Cureus, 2025; Academicmed, 2023).

The technical difficulties associated with developing natively in a Java-based Android environment for complex image processing tasks are very high, due to the hardware and computational requirements associated with such tasks. As a result, developers are unable to utilize the significant resources of well-established and well-documented computer vision libraries that exist within the Python programming language. Therefore, when developing applications for augmented reality (AR) or gesture-based control, developers have limited choices; to be successful in the Java environment, they must sacrifice some of the processing resources that they have available, or they must develop two different systems that will create significant challenges to integrate.

In particular, the following issues are addressed by this project:

- Lack of motion-based interaction with AR-based mobile games promotes sedentary lifestyles by mobile gamers;
- Building Java-only real-time hand tracking or color marker tracking solutions is computationally expensive and lacks reusable optimized computer vision libraries;
- There is little or no open-source documentation of efficient image processing implemented in Python directly as part of a game's Android application loop; and
- Existing AR-based mobile games typically rely on only one of the following.

V. Objectives

General Objective:

Using Python and the OpenCV library, create a real-time detection system that uses a camera on a mobile phone to recognize hand gestures and track colored objects. The detection system will accept live video frames from the camera and output the X and Y coordinates of all detected objects so that they may be used in an augmented-reality (AR) arcade-style mobile game using Android.

Specific Objectives

1. The development of a computer vision pipeline using OpenCV will allow us to detect the player's hand or color marker through color segmentation by converting our video frames into the HSV (Hue, Saturation, Value) domain and applying a threshold to these values.

2. Once the object has been isolated from the background using a contour detection and centroid calculation algorithm, its X and Y coordinates can be determined while the video frames are being processed in real-time.

3. Every video frame processed will also go through a series of image processing steps such as Gaussian blurring, morphological operations, and normalizing the image for consistent detection under multiple lighting conditions.

4. Low-latency communication will be established between the image processing module (via Python) and the Java frontend of the application we will build on Android using either the Chaquopy SDK or through a local socket server.

5. To evaluate the performance of this system, we will measure the accuracy of object detection, FPS (frames per second) of each video frame processed, and responsiveness of the system in all testing environments under varying lighting conditions.

VI. Proposed Solution

The proposed system uses an image processing module within the AR Motion Smasher mobile application, written as Python code. This module will execute repeatedly and will consume raw camera frames captured through the Android application (Java) as inputs, and generate centroid coordinates for each frame processed as outputs.

The solution works in the following way:

1. Supply: An Android application collects one video image from its front-facing camera using the CameraX API to pass to the Python component.

2. Process: The Python application transforms the BGR image into an HSV image, applies a color filter to extract an object, reduces noise with Gaussian Blur and Morphological Operations, locates all object outlines and measures the center of the biggest outline.

3. Produce: The centroids (X/Y) will then be sent back from the Python component to the Java application to determine if there are point-producing collisions with falling game objects displayed on the device's screen and thus adjust point totals and lives.

The uniqueness of the solution is the hybrid software approach. It eliminates the need for all computation to be done in Java on the device, where the Python component performs the highly intensive compute portion of image processing using an industry-standard computer vision algorithm, while the Java component performs the user interface, game execution, and rendering. This division of labor results in increased resource availability and a correspondingly smoother user experience.

VII. Scope and Limitations

Scope:

1. This project will include image processing through object's location detection and tracking of an object using the front camera live feed (Video stream) on your Android mobile device. The system will only track one object (either the person's hand or a marker with a specific color) at a time.

2. The program will use fixed values of HSV to perform color segmentation; however, the user can modify these values prior to the start of the game.

3. The system will provide real-time returns of the object's centroid coordinates in X,Y (horizontal and vertical coordinates) at 15 frames/second or more to the application.

4. The front end of the Android Java application will communicate with either a Chaquopy SDK or lightweight local Python Socket Server.

5. The testing of the system will take place on Android operating systems 8.0 or greater (Oreo).

Limitations:

1. The system is designed for indoor environments with good and stable lighting (poor lighting, direct sunlight, changing light conditions will degrade performance).
2. Only one object or hand can be tracked or detected at any given time (tracking or detecting multiple hands is not within the scope of this project).
3. The color segmentation approach will only work if the player's hand or marker is distinctly colored and visibly separated from the background (cluttered and or complex backgrounds will decrease detection accuracy).

4. Frame rates are limited based on hardware constraints, the design and stitching of camera videos, lighting conditions.

The order of operation for the image processing system is to break down complex tasks into several sequentially processed frames, with the primary purpose of converting the camera's "raw" video signal into usable spatial coordinates for each frame in the processing sequence. Below are the major steps in this process.

VIII. Methodology

System Workflow

Workflow defined below:

1. **Image Acquisition** - Captures a video frame using the Android CameraX API from the front-facing camera and sends its "raw" image data to the Python-based image processing module.
2. **Preprocessing** - Resizes the incoming image so that it matches a fixed resolution (e.g., 640x480 pixels) and converts it from BGR Colour Space to HSV Colour Space before applying a Gaussian Blur Filter (kernel size of 5x5) to reduce high-frequency noise.
3. **Segmentation** - Applies an already-defined colour mask (based on HSV Colour Co-ordinate) to the incoming image so that it creates a binary mask that isolates the target colour region from the rest of the frame (using the cv2.inRange() method).
4. **Morphological Operations** - These use the binary image mask produced in Step 3 to remove small noise artifacts from the cleaned binary mask by performing erosion followed by dilation (or "opening") operations.
5. **Contour Detection** - Cancels spurious contours that are detected using the cv2.findContours() method from the cleaned binary mask. Selects the largest contour area as the spatial target object.

6. **Feature Extraction/Centroid Calculation** - Uses cv2.moments() to calculate the X and Y coordinates of the centroid based on the calculated contour from Step 5.

Image Processing Techniques to be Used

The following will be used as image processing techniques along the way of a pipeline:

- Resize and Normalize Images- All frames will be resized to have a frame size of 640 x 480 pixels before processing. This will provide a consistent amount of computational load and coordinate output scale.
- Color Space Conversion- All frames will be converted from the default OpenCV color space (BGR) to the HSV color space (Hue – Saturation – Value), which provides a more robust method of performing color segmentation under differing brightness levels than does the RGB color space.
- Gaussian Blur (Filtering)- A Gaussian blur will be applied to each frame (using a 5 x 5 Gaussian kernel) prior to segmentation to decrease noise and increase the quality of the mask.
- HSV Thresholding (Segmentation)- The upper and lower boundary masks of the HSV color space will be applied together using cv2.inRange() to produce a binary image that contains all white pixel values for pixels in the area of interest and all black pixel values for non-area of interest pixels.
- Morphological Operations- Erosion and dilation operations will be performed sequentially (morphological opening) to eliminate salt-and-pepper noise from the binary mask and close small holes in detected regions.
- Contour Detection- cv2.findContours() will be used to locate the highest detected region's convex hull using the

RETR_EXTERNAL mode, and only the maximum area convex hull will be retained.

- Centroid Calculation Using Image Moments- The centroids cX and cY of the convex hull selected using cv2.moments() and the convex hull selected using cv2.findContours().

Tools and Technologies

- Languages used within the app programming: Python 3.10+ (image processing module), Java (Android game frontend).
- Development platforms used to build the app: Android Studio (Java frontend), VS Code/PyCharm (Python module).

- Core libraries:

  - OpenCV (cv2) — Core library used for performing image processing and to carry out image processing tasks such as frame capture, colour conversion, thresholding, contour detection and calculating moments.
  - NumPy — Library used to create and manipulate the array representing the data from the frames and perform mask operations.
  - Matplotlib — Used as a development tool to assist with visualising histograms, masks and overlays of frames.
  - Flask/Python Socket (socket module) — Component used to establish the local communication bridge between Python and Java if using Chaquopy is not an option.

- Integration SDK used to link the two programming languages on Android: Chaquopy (Python-in-Android plugin for Gradle) or Python socket server (Via local Wi-Fi/ localhost).

- Hardware used: Android smartphone (minimum Android 8.0) with front-facing camera or a development workstation used for testing of the Python module prior to deployment.

IX. Dataset / Data Collection

The Image Processing System (IPS) will use both a mixture of manually captured images and controlled video test samples as its basis to provide training and validation data. The color segmentation approach used by the image processing system does not require the collection of traditional labeled datasets for machine learning classifiers. The measured data will be used for calibration and validation in different illumination environments and with various backgrounds.

Image Collection

The team will use their Android device to capture video recordings of color markers and hand gestures at least three different ways (outdoor fluorescent, outdoor natural light, and low-light condition).

It is estimated that at least 200 validation frame images will be captured from these recordings that can be used to evaluate detection accuracy, false positive detection rate, and how accurately the centroid of the hand is located.

Data Characteristics

Number of Images: Approximately 200-300 frames data type image frames with RGB (640 x 480) resolution at 30 frames per second from live recordings of video.

There will be a minimum of three colors (e.g., red, green, orange) tested to determine how much difference there is between the different colors used in the photos.

The Steps To Accomplish The Data Preparation

1. Extract frames from the video recordings with OpenCV cv2.VideoCapture ().
2. Manually annotate the centroid position of all images with a subset of frames to see how accurately the centroids were located.
3. Perform range calibration for the different colors used with a trackbar

X. System Design

Architecture Diagram

The two-tier client-processing architecture is employed with the system. The client is the Android Java application that captures camera frames and displays the gaming interface. The processing layer is the Python module that receives frames and returns coordinate information. Internal API bridge (Chaquopy) and/or local socket connection provide communication between the two tiers of the architecture.

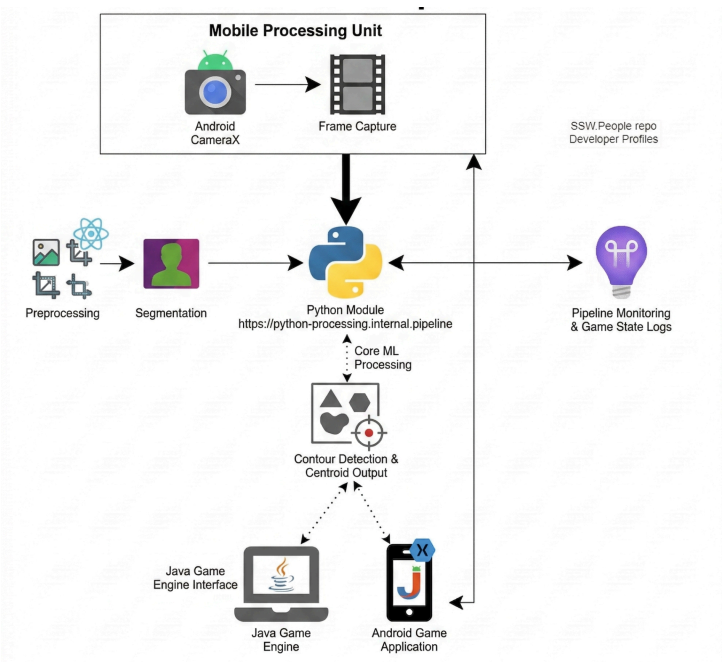

Figure 1. Architecture Diagram

Interface Design

Figure 2.1 Debug View

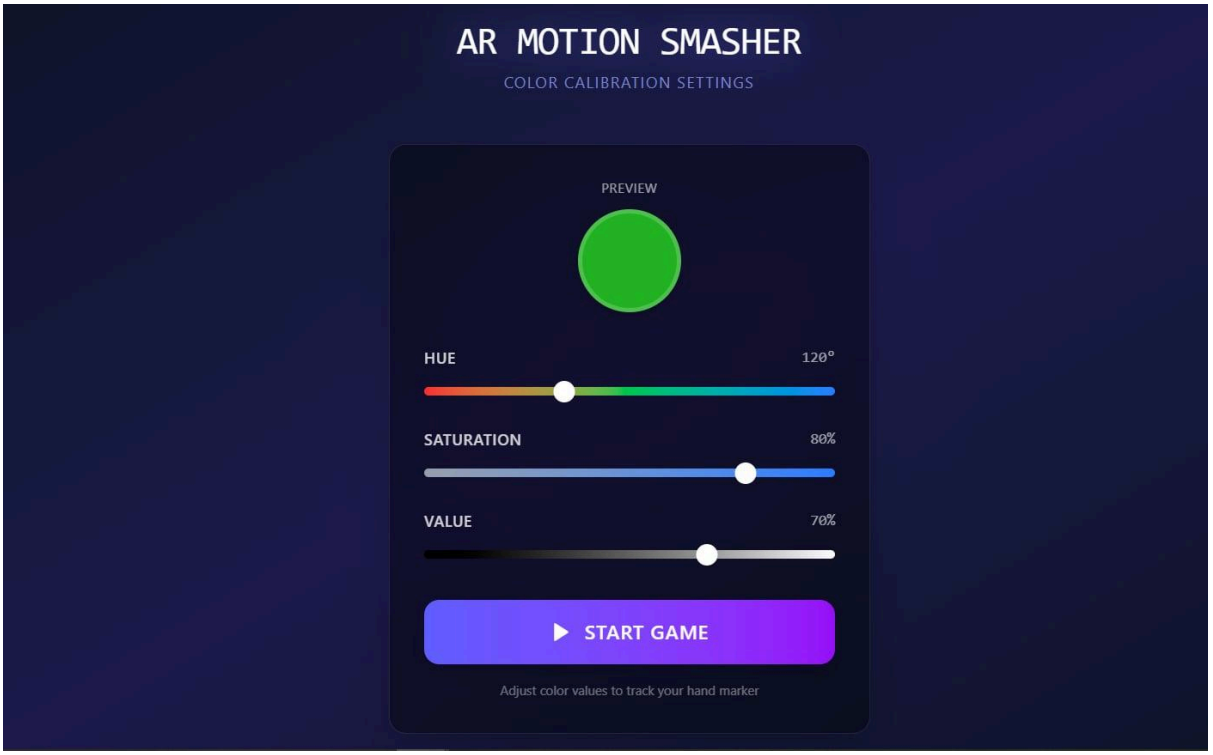

Figure 2.2 Color Calibration Settings Interface

Figure 2.3 Actual game interface

There are two visual interfaces exposed by this system:

Game Screen (User-Facing): Displays a live camera feed as the background of the game, along with 2D falling objects (i.e. meteors or geometric shapes) being rendered over the camera feed using Android Canvas/SurfaceView. When a user's hand is tracked, a small circular marker will be drawn at the centroid coordinates returned to provide visual feedback to the player on their tracked hand position in near real-time. The score and remaining number of lives will also appear at the top corners of the screen.

Debug View (Developer-Facing): This is a secondary display mode that is only available during development. It is the processed binary mask, contour outline, and centroid marker superimposed on top of the captured frame rendered in Python with Matplotlib or OpenCV.

XI. References

A. Bin Tomi and D. R. A. Rambli. 2011. A conceptual design for augmented reality games using motion detection as user interface and interaction. In *Visual Informatics: Sustaining Research and Innovations (IVIC 2011)*. Lecture Notes in Computer Science, vol. 7067. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-25200-6_29

S. Dhamodaran, et al. 2024. Implementation of hand gesture recognition using OpenCV. *International Journal of Research Publication and Reviews* 5, 5 (2024), 8039–8050.

S. Kim, et al. 2024. ARGo: Augmented reality-based mobile Go stone collision game. *Virtual Reality*. Springer. https://doi.org/10.1007/s10055-023-00919-4

M. C. Parashar, et al. 2020. Real-time hand gesture recognition system for controlling gaming applications using OpenCV and TensorFlow. *International Journal for Multidisciplinary Research (IJFMR)*.

T. A. Vu, et al. 2025. Integrating hand tracking technology for enhanced hand rehabilitation: A Flappy Bird game approach. In *Advances in Data Science and Optimization of Complex Systems (ICAMCS 2024)*. Lecture Notes in Networks and Systems, vol. 1311. Springer, Cham. https://doi.org/10.1007/978-3-031-90606-0_34

M. Wang, et al. 2025. Effectiveness of gamification interventions to improve physical activity and sedentary behavior in children and adolescents: Systematic review and meta-analysis. *JMIR Serious Games* 13 (2025). https://doi.org/10.2196/68151

OpenCV Team. 2024. OpenCV documentation—Image processing in OpenCV. Retrieved from https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html

Google LLC. 2024. Android CameraX API reference. Android Developers. Retrieved from https://developer.android.com/training/camerax

Chaquopy Documentation. 2024. Chaquopy: The Python SDK for Android. Retrieved from https://chaquo.com/chaquopy/doc/